

INTRODUCTION :

What is Event Streaming:

Event streaming is the digital equivalent of the human body's central nervous system.

Event streaming is the practice of capturing data in real-time from event sources like databases, sensors, mobile devices, cloud services, and software applications in the form of streams of events. Storing these event streams durably for later retrieval, manipulating, processing, and reacting to the event streams in real-time as well as retrospectively and routing the event streams to different destination technologies as needed.

Event streaming thus ensures a continuous flow and interpretation of data so that the right information is at the right place, at the right time.

Use Cases

To process payments and financial transactions in real-time, such as in stock exchanges, banks, and insurances.

To track and monitor cars, trucks, fleets, and shipments in real-time, such as in logistics and the automotive industry.

To continuously capture and analyze sensor data from IoT devices or other equipment, such as in factories and wind parks.

To collect and immediately react to customer interactions and orders, such as in retail, the hotel and travel industry, and mobile applications.

To monitor patients in hospital care and predict changes in condition to ensure timely treatment in emergencies.

To connect, store, and make available data produced by different divisions of a company.

To serve as the foundation for data platforms, event-driven architectures, and microservices.

Apache Kafka® is an event streaming platform.

Kafka combines three key capabilities so you can implement your use cases for event streaming end-to-end with a single battle-tested solution.

1. To publish (write) and subscribe to (read) streams of events, including continuous import/export of your data from other systems.

2. To store streams of events durably and reliably for as long as you want.

3. To process streams of events as they occur or retrospectively.

And all this functionality is provided in a distributed, highly scalable, elastic, fault-tolerant, and secure manner.

Kafka can be deployed on bare-metal hardware, virtual machines, and containers, and on-premises as well as in the cloud.

How does Kafka Works :

Kafka is a distributed system consisting of servers and clients that communicate via a high-performance TCP network protocol.

Servers: Kafka is run as a cluster of one or more servers that can span multiple datacenters or cloud regions.

Some of these servers form the storage layer, called the brokers.

Other servers run Kafka Connect to continuously import and export data as event streams to integrate Kafka with your existing systems such as relational databases as well as other Kafka clusters.

if any of its servers fails, the other servers will take over their work to ensure continuous operations without any data loss.

Clients: They allow you to write distributed applications and microservices that read, write, and process streams of events in parallel, at scale, and in a fault-tolerant manner even in the case of network problems or machine failures.

Main Concepts and Terminology :

An event records the fact that “something happened” in the world or in your business. It is also called record or message in the documentation.

When you read or write data to Kafka, you do this in the form of events. Conceptually, an event has a key, value, timestamp, and optional metadata headers.

Producers are those client applications that publish (write) events to Kafka, and consumers are those that subscribe to (read and process) these events.

In Kafka, producers and consumers are fully decoupled and agnostic of each other, which is a key design element to achieve the high scalability that Kafka is known for.

Events are organized and durably stored in topics.

A topic is similar to a folder in a filesystem, and the events are the files in that folder.

Topics in Kafka are always multi-producer and multi-subscriber: a topic can have zero, one, or many producers that write events to it, as well as zero, one, or many consumers that subscribe to these events.

Events in a topic can be read as often as needed-unlike traditional messaging systems, events are not deleted after consumption.

Events in a topic can be read as often as needed-unlike traditional messaging systems, events are not deleted after consumption.

Kafka's performance is effectively constant with respect to data size, so storing data for a long time is perfectly fine.

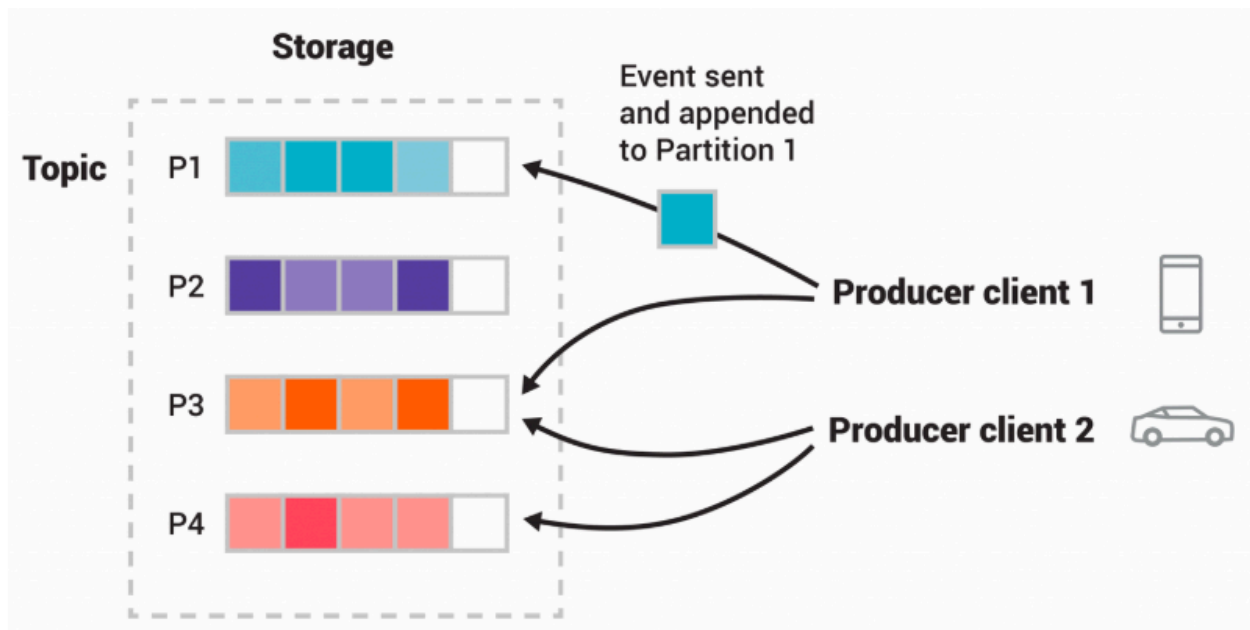
Topics are partitioned , meaning a topic is spread over a number of "buckets" located on different Kafka brokers.

This distributed placement of your data is very important for scalability because it allows client applications to both read and write the data from/to many brokers at the same time.

When a new event is published to a topic, it is actually appended to one of the topic's partitions.

Events with the same event key are written to the same partition.

Kafka guarantees that any consumer of a given topic-partition will always read that partition's events in exactly the same order as they were written.



To make your data fault-tolerant and highly-available, every topic can be replicated, even across geo-regions or datacenters, so that there are always multiple brokers that have a copy of the data just in case things go wrong, you want to do maintenance on the brokers, and so on.

A common production setting is a replication factor of 3, i.e., there will always be three copies of your data. This replication is performed at the level of topic-partitions.

Kafka API's :

Kafka has five core APIs.

1. The Admin API to manage and inspect topics, brokers, and other Kafka objects.

2. The Producer API to publish (write) a stream of events to one or more Kafka topics.

3. The Consumer API to subscribe to (read) one or more topics and to process the stream of events produced to them.

4. The Kafka Streams API to implement stream processing applications and microservices.

It provides higher-level functions to process event streams, including transformations, stateful operations like aggregations and joins, windowing, processing based on event-time, and more.

Input is read from one or more topics in order to generate output to one or more topics, effectively transforming the input streams to output streams.

5. The Kafka Connect API to build and run reusable data import/export connectors that consume (read) or produce (write) streams of events from and to external systems and applications so they can integrate with Kafka.